QuadCore at ArGuard Shared Tasks: A Simple Threshold Calibration Approach for Arabic Hateful Meme Detection

Umme Saima Tonny, Maimuna Tabassum, Sidratul Muntaha, Sabila Anzum
ummesaiama000@gmail.com, maimunatabassum2002@gmail.com,
smuntaha226@gmail.com, sabilaa777@gmail.com
Department of Computer Science and Engineering,
Chittagong University of Engineering & Technology, Chattogram, Bangladesh

Abstract

We describe our system for Subtask A1 (binary hateful meme detection) of the ArGuard 2026 shared task (Alam et al., 2026). Our final system fine-tunes MARBERTv2 with class-weighted loss and macro-F1-based checkpoint selection, then applies a post-hoc decision-threshold calibration to correct for a class-distribution shift we found between the training/development splits (37.8% Hateful) and the held-out test split (29.6% Hateful). This calibration step requires no retraining and adds 3.3 macro-F1 points (0.7286 → 0.7616) on the official test split. On the same test split, this calibrated system also outperforms two alternatives we evaluated there – multi-seed ensembling and naive multimodal late-fusion. They match or exceed backbone-substitution and multi-model-ensemble variants that we were only able to evaluate on local development data. We report our full experimental log, including the negative results, with each result explicitly labeled by evaluation split, and compare our score to the AHA-Memes benchmark, where a much larger vision-language model reaches a similar range. Code and configuration files are released for reproducibility.[1]

1 Introduction

Hate speech in memes is encoded in an image and its overlaid text, and often leans on sarcasm, humor, or culturally specific references that are hard to catch from either modality alone. ArGuard 2026 (Alam et al., 2026) targets this problem for Arabic, building on the AHA-Memes benchmark (Kmainasi et al., 2026), and Subtask A1 asks for a binary Hateful / Not Hateful label given a meme's OCR-extracted text.

We focus on two things: class-weighted loss to handle label imbalance, and post-hoc threshold calibration to correct a train/test distribution shift we noticed during development. We also report what did not work: several natural extensions that looked promising on local validation data but failed to transfer to the real test set, since that is useful information for anyone building on this task next.

[1] https://github.com/Saima040/arguard-2026-hateful-meme

2 Related Work

Our work is based on the ArGuard 2026 overview paper (Alam et al., 2026) and the AHA-Memes benchmarking dataset (Kmainasi et al., 2026) which it describes, while positioning our approach as lightweight in comparison with vision-language methods explored in ArGuard 2026.

Earlier Arabic systems for detecting offensive language include Abu Farha and Magdy (2020), who proposed a joint CNN-BiLSTM model for predicting hate speech, offensive language and sentiment and outperformed single-task systems on OSACT4, and Abuzayed and Elsayed (2020), who compared 15 classical and neural approaches in a deliberately lightweight setup. Surveys covering the broader topic highlight dialectal diversity and lack of benchmark data as important issues (Husain and Uzuner, 2021; Alhazmi et al., 2024; Alshahrani et al., 2025; Al-Saqqa et al., 2024; Abdelsamie et al., 2024).

Among the existing corpora, Duwairi et al. (2021) provided ArHS with 9,833 tweets and CNN/CNN-LSTM/BiLSTM-CNN models in binary, ternary and multiclass classification tasks; Ahmad et al. (2024) introduced the much bigger Jordanian Hate Speech Corpus with 403,688 tweets; and Charfi et al. (2024) introduced ADHAR, a dialect-balanced and considerably smaller corpus, where a fine-tuned AraBERT achieves 94% accuracy/F1 – but only on plain-text tweets, a simpler task than ours.

Most closely related to our work is Kmainasi et al. (2026), whose AHA-Memes benchmark measures the performance of purely textual and vi-

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 490–496
October 28-29, 2026 ©2026 Association for Computational Linguistics

Split	N	Hateful	% Hate
train	3,500	1,324	37.8%
dev	500	189	37.8%
dev_test	500	189	37.8%
test	500	148	29.6%

Table 1: Dataset splits and label distribution. `dev_test` is the unlabeled leaderboard split during the Development Phase, while `test` is the blind, triple-annotated split used for final, official scoring during the Testing Phase.

sual, naive late fusion, and vision-language model (VLM) approaches on Arabic memes. The best of their results – a LoRA-fine-tuned Qwen3-VL-8B – reaches 0.768 macro-F1; we return to this comparison in Section 7.

Our system builds on three public Arabic BERT-family encoders: MARBERTv2 (Abdul-Mageed et al., 2021) (dialectal, social-media Arabic), AraBERT (Antoun et al., 2020) (mostly MSA news/Wikipedia text), and CAMeL-BERT (Inoue et al., 2021) (a mix of MSA, dialectal, and classical Arabic). We compare all three in Section 6.2. Our multimodal experiment additionally uses BEiT (Bao et al., 2022) as the image encoder.

3 Data

We used the official ArGuard 2026 Task 1 / Track A dataset (QCRI/ArGuard-Task1), available on Hugging Face.[2] Table 1 shows the official splits.

The shared task had two phases. During the Development Phase, we trained our models on the `train` set, used the labeled `dev` set for validation, and submitted predictions on the unlabeled `dev_test` split for a public leaderboard. Once that phase closed, `dev`'s gold labels were released and it no longer served as a separate leaderboard-facing validation set. For the **Testing Phase**, submissions were evaluated on a completely blind `test` split. To make full use of available labeled data for this final submission, we combined `train` and `dev` into one training pool. From this combined pool, we create a fresh stratified 10% internal validation slice (Section 5). We used this new slice strictly for checkpoint selection and threshold calibration, rather than sticking to the Development Phase setup of training on `train` and validating on `dev`.

The test split has a noticeably lower Hateful rate

[2] https://huggingface.co/datasets/QCRI/ArGuard-Task1

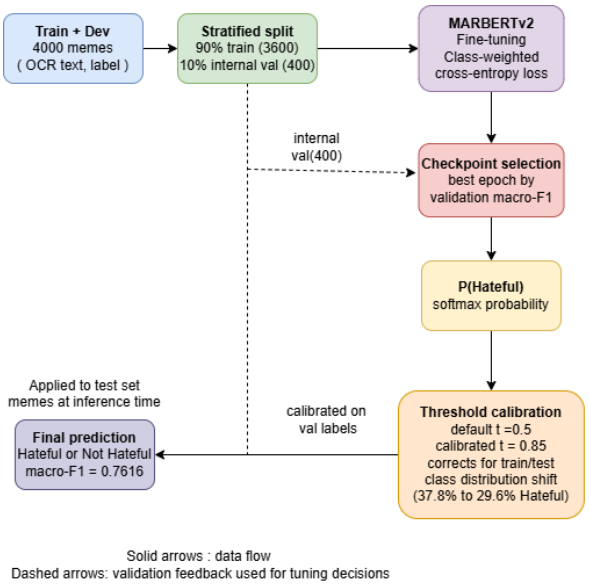


Figure 1: System pipeline. Dashed arrows show where the internal validation split feeds into checkpoint selection and threshold calibration.

(29.6%) than train, dev, and dev_test (all 37.8%). We found this gap empirically, by comparing our own model's aggregate predictions on the unlabeled test set against the known test label distribution released after the Testing Phase closed; it is not stated anywhere in the task documentation, and it is what motivates the threshold calibration in Section 4.3. We use no data beyond the official release, and no preprocessing beyond standard subword tokenization.

4 System Overview

Figure 1 shows the pipeline: MARBERTv2 is fine-tuned on train+dev with class-weighted loss, the checkpoint is chosen by validation macro-F1, and the resulting probabilities go through a calibrated threshold to produce the final label.

4.1 Base Model and Training

We fine-tune MARBERTv2 (Abdul-Mageed et al., 2021) for binary classification. The OCR-extracted text goes to the encoder, and the pooled `[CLS]` vector is sent to two logits through a linear head. We chose MARBERTv2 after comparing it with AraBERT and CAMeL-BERT (Section 6.2). MARBERTv2 is suitable because it was trained on social networks and dialects, which is similar to the kind of language used in memes. The other options are more formal, so they do not match meme text as well as MARBERTv2 does.

In our submission, we combine all the train and dev examples, giving 4,000 examples. We hold back 400 examples, or 10%, as an internal validation set to select the best model, so our final model can use more data than a model trained only on the original training set, which is important to consider when comparing with other systems.

4.2 Class-Weighted Loss

Training data is not balanced (37.8% Hateful, 62.2% Not Hateful), so we use weighted cross-entropy instead of standard cross-entropy. For each class c, it gets weight $w_c = N/(2n_c)$, where n_c is its training-set count and N the total. Since Hateful examples are less common, the model gets a bigger penalty when it incorrectly classifies them. This discourages the model from always predicting the majority class. This change substantially improved the Hateful-class F1 score compared to the unweighted method. We also choose model checkpoints based on validation macro-F1 instead of loss because macro-F1 is the official evaluation metric for the task.

4.3 Post-Hoc Threshold Calibration

According to the usual rule, something is Hateful whenever $P(\text{Hateful}) \geq 0.5$. Since our model is trained on data that is 37.8% Hateful, that default threshold is implicitly tuned to that ratio – but the real test set is only 29.6% Hateful, so we suspected it would over-predict Hateful on test data.

We checked: at $t = 0.5$, the model predicted that 39.8% of the test data was Hateful, while the true rate was 29.6%. We then tested different threshold values from 0.05 to 0.95, in steps of 0.01, using the labeled internal validation data described in Section 5. We selected the value that gave the highest macro-F1 score on that validation split. This search located an approximate optimum around $t = 0.85$. We then confirmed this value using the official test-set leaderboard itself, by submitting predictions at $t = 0.74$, $t = 0.85$, and $t = 0.92$ and comparing their official macro-F1 scores (0.7572, 0.7616, 0.7610 respectively; see Table 3). We want to be transparent that the final threshold was selected after trying a small number of options (three) on the test set, rather than using only the validation data. The validation results had already pointed us toward the right direction and approximate range, but choosing the exact value of $t = 0.85$ using the test set is still a limitation. This means we cannot be fully certain that $t = 0.85$ would work equally well on a different test split or dataset. Importantly, this process did not involve retraining the model. We only used the probabilities that the model had already produced.

5 Experimental Setup

Our final model was trained using the combined train and development data, giving us 4,000 examples in total. From this combined set, we kept a stratified 10% split (400 examples) for checkpoint selection and threshold calibration, as described in Section 3. The earlier Development-Phase runs, marked with † in Table 3, were trained only on the `train` set and evaluated on the `dev_test` leaderboard split. The runs marked with * were evaluated only on our own local validation split and were not submitted to the leaderboard.

We fine-tuned the model for up to 6 epochs using a batch size of 16, learning rate of 2×10^{-5}, weight decay of 0.01, and a linear warmup over the first 10% of training steps. The maximum sequence length was set to 256. We started with standard BERT fine-tuning settings and also tried adjusting these settings once (Table 3), but the modified settings performed worse, so we kept the original defaults. We used HuggingFace `transformers` and `PyTorch` for the model and `scikit-learn` for evaluation metrics and stratified splitting. All training was done on a single NVIDIA T4 GPU with 16GB memory through a free-tier Kaggle notebook, with each run taking roughly 10–12 minutes.

Our submitted result uses seed 42. We also checked whether the result was reliable by running the model with a 3-seed ensemble (seeds 42, 7, 123), which did not beat the single-seed result. We did not otherwise run repeated trials of the same configuration, so our reported numbers may understate the true run-to-run variance.

6 Results

6.1 Final System Performance

Our submitted system – MARBERTv2 with class-weighted loss, trained on train+dev, with calibrated threshold $t = 0.85$ – scores as shown in Table 2 on the official, triple-annotated `test` split. This is our official result. An earlier, uncalibrated version of this pipeline scored 0.6875 macro-F1 on the `dev_test` leaderboard during the Development Phase (rank 12 of 19 teams).

Metric	Value
Macro-F1 (official)	**0.7616**
Accuracy	0.7960
Macro precision	0.7559
Macro recall	0.7690
Weighted F1	0.7985
F1 (Hateful)	0.6710
F1 (Not Hateful)	0.8522

Table 2: Final system performance on the official `test` split.

Hateful-class F1 (0.6710) trails Not-Hateful F1 (0.8522) even after weighting and calibration – in line with the wider literature's finding that hate speech is harder to catch than its absence, given its lower frequency and greater linguistic variety (sarcasm, indirect reference, dialect).

6.2 Full Experimental Log and Ablation

Table 3 shows all the configurations we tested, including the ones that did not work well. Each row clearly indicates which evaluation split was used. We include the complete experiment log because some changes that performed well on one split did not perform as well on the official test split, as discussed in Section 7.

The two Development-Phase rows (†) show that switching from unweighted AraBERT to class-weighted MARBERTv2 improved macro-F1 by +0.0457 on the `dev_test` leaderboard split. We then trained on the combined train+dev data and, before calibration, achieved a macro-F1 of 0.7286 on the official `test` split. Applying threshold calibration without changing or retraining the model gave another +0.033 macro-F1. It was the largest and simplest improvement in our pipeline. However, as discussed in Section 4.3, the exact threshold was chosen after a small number of evaluations on the test set, so this gain should be interpreted with some caution.

Replacing MARBERTv2 with CAMeL-BERT or changing the learning rate and number of epochs both performed worse than the original setup on our local validation data. We also tested different weights for a three-model ensemble combining MARBERTv2, AraBERT, and CAMeL-BERT. The whole search was done only on the local validation split and eventually assigned 100% of the weight to MARBERTv2, so the ensemble did not provide any improvement there.

We also tested a simple multimodal approach that combined MARBERTv2 and BEiT by concatenating their outputs and passing them through an MLP head. Although this model achieved a reasonably good local validation score of 0.7211*, it scored only 0.6550† on the Development-Phase leaderboard (`dev_test` split), which was worse than the text-only model at that stage. This result is consistent with the finding of Kmainasi et al. (2026), who reported that simple late-fusion approaches can perform worse than both text-only fine-tuned models and properly fine-tuned vision-language models.

Approach	F1
AraBERT, unweighted loss	0.6418†
MARBERTv2, class-weighted, macro-F1 sel.	0.6875†
CAMeL-BERT (backbone swap)	0.7194*
MARBERTv2, retuned LR/epochs	0.7097*
3-way ensemble (MARBERT+AraBERT+CAMeL)	0.7374*
MARBERTv2 + BEiT late-fusion multimodal	0.6550†
3-seed ensemble, no calibration	0.6809
MARBERTv2 on train+dev, default t	0.7286
+ threshold calibration (t=0.74)	0.7572
+ threshold calibration (t=0.85)	**0.7616**
+ threshold calibration (t=0.92)	0.7610
3-seed ensemble + median threshold	0.7516
3-seed ensemble + fixed t=0.85	0.7576

Table 3: Full experimental log. * = scored only on our local internal validation split, never submitted to any leaderboard. † = official Development-Phase leaderboard score (`dev_test` split). Unmarked rows = official Testing-Phase leaderboard score (blind `test` split).

Finally, during the Testing Phase, we tried combining predictions from multiple random seeds with threshold calibration. This did not improve over the single-seed calibrated model, whether we used the median threshold for each seed or a fixed threshold of 0.85. This suggests that, for this particular approach, the calibrated single-model setup may already be close to its best performance, although our results alone are not enough to prove that it is a true local optimum.

6.3 Error Analysis

After the gold test labels became publicly available, we manually reviewed all 58 false positives and 44 false negatives from our final system. We found four recurring types of errors: OCR corruption sometimes led to incorrect predictions in both directions; implicit misogyny was sometimes missed when no explicit slur was present; strong or violent words used in a non-hateful, colloquial way

were sometimes incorrectly flagged as hate speech; and politically or symbolically charged references were occasionally mistaken for hateful content. We did not observe any clear pattern suggesting that the errors were concentrated in a particular Arabic dialect. However, we did not perform a formal dialect-level analysis, so this observation should be treated cautiously. Appendix A provides examples of each of these error patterns.

7 Discussion

The main lesson here is a simple one: on the official test split, a well-motivated post-processing fix (threshold calibration) outperformed the architectural changes we were able to evaluate on that same split – multi-seed ensembling and naive multimodal late-fusion. We also compared backbone substitution and multi-model ensembling against our baseline, but only on local validation data, so those comparisons are not on an equal footing with the test-split results and should be read with that caveat. We think the architectural extensions that we did test on local validation data may have partly reflected validation-set overfitting, a real risk whenever development decisions are made repeatedly against the same small (400–500 example) split.

Against Kmainasi et al. (2026), whose LoRA-fine-tuned Qwen3-VL-8B reaches 0.768 macro-F1 on the AHA-Memes benchmark that this shared task's data is drawn from, our 0.7616 lands within half a point. We note two caveats on this comparison: first, as stated in Section 5, our system was trained on train+dev (4,000 examples), which may give it access to more labeled data than a system trained on train alone; and second, we have not independently confirmed that the 0.768 figure is computed on the exact same `test` split used for our official score, only that it is drawn from the same underlying benchmark. With these caveats, we read our result as suggestive evidence that a lightweight, calibrated text-only encoder can be competitive with substantially larger vision-language approaches on this task, rather than as a firm claim about a ceiling for the task in general; more labeled or silver-labeled data, or better joint image-text pretraining, could plausibly do better.

Our calibration was discovered empirically, not from any documented task specification, so it is specific to this particular shift and would need rederiving if the test distribution changes in future task runs.

8 Conclusion

We reach 0.7616 macro-F1 on ArGuard 2026 Subtask A1 with class-weighted MARBERTv2 fine-tuning and post-hoc threshold calibration. We report the full experimental log, including negative results (backbone swaps, ensembling, naive multimodal fusion) with each result explicitly labeled by evaluation split, and find our result competitive with a substantially larger vision-language system from concurrent work, with the caveats discussed in Section 7. Future work could explore silver-label pretraining or more careful multimodal fusion, though our results suggest naive fusion alone is not enough to help on this task.

Limitations

Our calibration threshold was finalized with the help of a small number of test-set evaluations (Section 4.3), which is a genuine limitation on how confidently it can be expected to generalize to new data drawn from a different distribution. Most of our configurations are also single-seed and single-split, aside from the explicit seed-ensemble check, so our reported numbers may understate true variance. Several of our comparisons (backbone substitution, multi-model ensembling) were only evaluated on local validation data rather than the official test split, so they are not directly comparable, split-for-split, to our final result.

Ethical Considerations

A hate-meme classifier like the one we created could be used in a way that is too broad when it comes to removing content. This could end up marking ways of speaking of certain religious groups or certain political ideas as harmful more than they should, especially if those groups appear too little or too much in the data used to train the model. Our system is something we built for a competition. It is not meant or checked for use in real-life moderation. We did not do a check for different ways of speaking in the data. The difference in how classes are spread out between training and testing data that we found (Section 3) is one clear reason for differences in the data. We cannot say for sure that there are not other ways of speaking, topics, or groups that are not shown enough and that our overall measurements would not catch.

The dataset is available to use under CC BY-NC 4.0 for non-commercial research (Kmainasi et al., 2026) and necessarily contains offensive content

by design; we used the provided examples only for training and evaluation within the scope of this shared task, and did not collect or redistribute any additional data.

Acknowledgments

We thank the ArGuard 2026 organizers for curating the dataset, running the evaluation infrastructure, and providing detailed guidance on writing this paper.

References

Mahmoud M. Abdelsamie, Samar S. Azab, and Hesham A. Hefny. 2024. A comprehensive review on Arabic offensive language and hate speech detection on social media: methods, challenges and solutions. *Social Network Analysis and Mining*, 14:111.

Muhammad Abdul-Mageed, AbdelRahim Elmadany, and El Moatez Billah Nagoudi. 2021. ARBERT & MARBERT: Deep bidirectional transformers for Arabic. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 7088–7105, Online. Association for Computational Linguistics.

Ibrahim Abu Farha and Walid Magdy. 2020. Multitask learning for Arabic offensive language and hate-speech detection. In *Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools, with a Shared Task on Offensive Language Detection*, pages 86–90, Marseille, France. European Language Resource Association.

Abeer Abuzayed and Tamer Elsayed. 2020. Quick and simple approach for detecting hate speech in Arabic tweets. In *Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools, with a Shared Task on Offensive Language Detection*, pages 109–114, Marseille, France. European Language Resource Association.

Ashraf Ahmad, Mohammad Azzeh, Eman Alnagi, Qasem Abu Al-Haija, Dana Halabi, Abdullah Aref, and Yousef AbuHour. 2024. Hate speech detection in the Arabic language: corpus design, construction, and evaluation. *Frontiers in Artificial Intelligence*, 7:1345445.

Samar Al-Saqqa, Arafat Awajan, and Bassam Hammo. 2024. A survey of hate speech detection for Arabic social media: Methods and datasets. *Procedia Computer Science*, 251:224–231.

Firoj Alam, Md. Rafiul Biswas, Mohamed Bayan Kmainasi, Ali Ezzat Shahroor, Hamdy Mubarak, Georgios Mikros, Abul Hasnat, and Wajdi Zaghouani. 2026. ArGuard Shared Task: Harmful content detection in Arabic Memes and LLM Prompts. In *Proceedings of the Fourth Arabic Natural Language Processing Conference: Shared Tasks*, Budapest, Hungary. Association for Computational Linguistics.

Ali Alhazmi, Rohana Mahmud, Norisma Idris, Mohamed Elhag Mohamed Abo, and Christopher Eke. 2024. A systematic literature review of hate speech identification on Arabic Twitter data: research challenges and future directions. *PeerJ Computer Science*, 10:e1966.

Eman S. Alshahrani, Mehmet S. Aksoy, and Ahmed Emam. 2025. Detection of hate speech and offensive language in Arabic text: A systematic literature review. *Applied Computational Intelligence and Soft Computing*, 2025:5565888.

Wissam Antoun, Fady Baly, and Hazem Hajj. 2020. AraBERT: Transformer-based model for Arabic language understanding. In *Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools, with a Shared Task on Offensive Language Detection*, pages 9–15, Marseille, France. European Language Resource Association.

Hangbo Bao, Li Dong, Songhao Piao, and Furu Wei. 2022. BEiT: BERT pre-training of image transformers. In *International Conference on Learning Representations*.

Anis Charfi, Mabrouka Besghaier, Raghda Akasheh, Andria Atalla, and Wajdi Zaghouani. 2024. Hate speech detection with ADHAR: a multi-dialectal hate speech corpus in Arabic. *Frontiers in Artificial Intelligence*, 7:1391472.

Rehab Duwairi, Amena Hayajneh, and Muhannad Quwaider. 2021. A deep learning framework for automatic detection of hate speech embedded in Arabic tweets. *Arabian Journal for Science and Engineering*, 46:4001–4014.

Fatemah Husain and Ozlem Uzuner. 2021. A survey of offensive language detection for the Arabic language. *ACM Transactions on Asian and Low-Resource Language Information Processing*, 20(1):Article 12.

Go Inoue, Bashar Alhafni, Nurpeiis Baimukan, Houda Bouamor, and Nizar Habash. 2021. The interplay of variant, size, and task type in Arabic pre-trained language models. In *Proceedings of the Sixth Arabic Natural Language Processing Workshop*, pages 92–104, Kyiv, Ukraine (Online). Association for Computational Linguistics.

Mohamed Bayan Kmainasi, Ali Ezzat Shahroor, Abul Hasnat, Md. Rafiul Biswas, Wajdi Zaghouani, and Firoj Alam. 2026. AHA-Memes: A fine-grained multimodal benchmark for understanding hate in arabic memes. *arXiv preprint arXiv:2607.27393*.

A Error Analysis Details

We manually reviewed all 58 false positives (predicted Hateful but actually Not Hateful) and 44 false negatives (predicted Not Hateful but actually Hateful) from our final system on the official test split. From this review, we found four recurring error patterns, which are summarized in Section 6.2 and described in more detail below.

OCR corruption: A noticeable number of false positives contain heavily corrupted OCR output instead of clear Arabic text. These examples often consist of broken or incomplete character sequences with little recognizable meaning. Since the text is difficult to interpret, the model may sometimes respond to these noisy patterns rather than to meaningful semantic content.

Implicit misogyny without explicit slurs (false negatives): We found several Hateful examples that targeted women through sexist or mocking statements without using explicit slurs. For example, one meme mocked women's ability to succeed in the workplace, another sarcastically suggested that a woman should turn to sex work because a respectable job would not allow her to afford a car and apartment, and another referred to body-shaming a public figure. These examples require understanding the context and cultural meaning of the message rather than relying on obvious hateful keywords. This may explain why the model had more difficulty detecting them.

Intense or violent language used colloquially (false positives): Some false positives contained strong or violent words, including war- or martyrdom-related slang and a metaphorical reference to a "dog", but the words were used jokingly or as part of an everyday expression rather than as a targeted attack. The model appears to sometimes associate this type of strong vocabulary with the Hateful class even when the context is not actually hateful.

Politically or symbolically charged references: We also found cases where politically sensitive or symbolic references were incorrectly classified as Hateful. For example, at least one false positive involved a comparison of national flags that appeared to be commentary rather than hate speech. This suggests that the model may sometimes associate politically or symbolically charged content with the Hateful class even when there is no direct attack against a protected group.

We did not observe a clear concentration of errors in any particular Arabic dialect. However, we did not perform a formal dialect-tagging analysis, so this observation should be treated cautiously. A more systematic dialect-level analysis would be useful in future work.